

No	Test Cases	Test Steps	Test Data (Request Body)	Expected Result	Actual Result	Status	Link to Bug Report
1	1. Verify the user can add an existing product ID to the kit.	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request by using an existing product ID in the path params and request body. 3. The user adds existing products to the kit on the response.	Path params: id=6 { "productsList": [{ "id": 3, "quantity": 3 }] }	- Added existing products to the kit - Status: 200 OK	- Added existing products to the kit - Status: 200 OK	Passed	
3	Verify the user cannot add a product with a negative product ID (-3) to the kit	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products 3. In the request body, set the product id to -3.	Path params: id=3 { "productsList": [{ "id": -3, "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- Request was accepted, no error message was displayed in the request body - Status: 200 OK	Failed	https://keltinfreelow.atlassian.net/browse/S10-33
4	Verify the user cannot add a product with a decimal product ID (1.5) to the kit.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products 3. In the request body, set the product id to 1.5.	Path params: id=3 { "productsList": [{ "id": 1.5, "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-34
5	Verify the user cannot add a product ID with Latin letters ("abc") to the kit	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request setting the product id to "abc" in the request body.	Path params: id=3 { "productsList": [{ "id": "abc", "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-35
6	Verify the user cannot add a product ID with special characters ("@#!") in the kit	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request setting the product id to "@#!" in the request body.	Path params: id=3 { "productsList": [{ "id": "@#!", "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-36
7	Verify the user cannot add a product ID with an empty string ("") in the kit	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request setting the product id to empty string "" in the request body.	Path params: id=3 { "productsList": [{ "id": "", "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-37
8	Verify the user cannot add a product to a kit if the "id" parameter is missing from the request body.	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request leaving the "id" field completely missing from the product object inside the body.	Path params: id=3 { "productsList": [{ "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-38
9	Verify the user cannot add a product ID with Non-Latin characters ("привет") in the kit	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request setting the product id to ("привет") in the request body.	Path params: id=3 { "productsList": [{ "id": "привет", "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-39
10	Verify the user cannot add a product with a product ID of 0 to the kit.	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request setting the product id to (0) in the request body.	Path params: id=3 { "productsList": [{ "id": 0, "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- Request was accepted, no error message was displayed in the request body - Status: 200 OK	Failed	https://keltinfreelow.atlassian.net/browse/S10-40
11	Verify the user cannot add a product with a non-existent product ID (9999) to the kit.	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request setting the product id to (9999) in the request body.	Path params: id=3 { "productsList": [{ "id": 9999, "quantity": 3 }] }	- Error message received - Status: 400 Bad Request	- Request was accepted, no error message was displayed in the request body - Status: 200 OK	Failed	https://keltinfreelow.atlassian.net/browse/S10-41
12	Verify the user cannot add products to a kit using a non-existent Kit ID (9999) in the URL path parameter.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products - Click the "Send" button	Path params: id = 9999 { "productsList": [{ "id": 1, "quantity": 3 }] }	- Error message displayed - Status: 404 Not Found	- Error message displayed - Status: 404 Not Found	Passed	
13	Verify the user cannot add products to a kit using a non-existent Kit ID (-3) in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (-3) 3. Ensure the request body has a valid product configuration.	Path params: id = -3 { "productsList": [{ "id": 1, "quantity": 3 }] }	- Error message displayed - Status: 404 Not Found	- Error message displayed - Status: 404 Not Found	Passed	
14	Verify the user cannot add products to a kit using a string (abc) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (abc) 3. Ensure the request body has a valid product configuration.	Path params: id = abc { "productsList": [{ "id": 1, "quantity": 3 }] }	- Error message displayed - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-23

15	Verify the user cannot add products to a kit using special characters (@#!) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (@#!) 3. Ensure the request body has a valid product configuration.	Path params: id = @#! <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the 'Send' button	- Error message received - Status: 404 Not Found	- Error message displayed - 405 Method Not Allowed	Failed	https://keltinfreelow.atlassian.net/browse/S10-25
16	Verify the user cannot add products to a kit when leaving the Kit ID completely empty in the URL path parameter.	1. Click on the Params tab in Postman. 2. Clear the value field entirely for the "id" path variable so it is blank. 3. Ensure the request body has a invalid product configuration.	Path Params: id = leave empty <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message received - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-26
17	Verify the user cannot add products to a kit using an excessively large numeric value (999999999999) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (999999999999) 3. Ensure the request body has a invalid product configuration.	Path params: id = 999999999999 <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message received - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-27
18	Verify the user cannot add products to a kit using a decimal value (3.5) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (3.5) 3. Ensure the request body has a invalid product configuration.	Path params: id = 3.5 <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message received - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-28
19	Verify the user cannot add products to a kit using a leading zero value (01) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (01) 3. Ensure the request body has a invalid product configuration.	Path params: id = 01 <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message received - Status: 400 Bad Request	- Error message displayed - Status: 400 Bad Request	Failed	https://keltinfreelow.atlassian.net/browse/S10-42
20	Verify the user cannot add products to a kit when the Kit ID path parameter consists only of a blank space character.	1. Click on the Params tab in Postman. 2. Clear the "id" value field, type a single space using your spacebar.	Path params: id = single space <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message received - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-29
21	Verify the user cannot add products to a kit using a zero value (0) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (0). 3. Ensure the request body has a invalid product configuration.	Path params: id = 0 <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre>	- Error message displayed - Status: 404 Not Found	- Error message displayed - Status: 404 Not Found	Passed	
22	Verify the user cannot add products to a kit using a URL-encoded special character (%23) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (%23). 3. Ensure the request body has a invalid product configuration.	Path params: id = %23 <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send button	- Error message received - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-30
23	Verify the user cannot add products to a kit using a path variable value that contains a forward slash (1/2) as the Kit ID.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (1/2). 3. Ensure the request body has a invalid product configuration.	Path params: id = 1/2 <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message displayed - Status: 404 Not Found	- Error message displayed - Status: 404 Not Found	Passed	
24	Verify the user cannot add products to a kit using a boolean value (true) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (true). 3. Ensure the request body has a invalid product configuration.	Path params: id = true <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message received - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-31
25	Verify the user cannot add products to a kit using a null value keyword (null) as the Kit ID in the URL path parameter.	1. Click on the Params tab in Postman. 2. Change the value of the "id" path variable to (null). 3. Ensure the request body has a invalid product configuration.	Path params: id = null <pre>{ "productsList": [{ "id": 1, "quantity": 3 }] }</pre> Click the "Send" button	- Error message received - Status: 404 Not Found	- The server crashed, displaying a HTML body - Status: 500 Internal Server Error	Failed	https://keltinfreelow.atlassian.net/browse/S10-32

26	Verify the user cannot add products to a kit when the quantity parameter value is a string ("five") instead of an integer.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products	<pre>{ "productsList": [{ "id": 1, "quantity": "five" }] }</pre> Click the "Send" button	- The response status code is 400 Bad Request - An error message in the response body, indicating invalid an quantity	The response status code is 400 Bad Request, but the response body incorrectly states "No more than 30 items per set".	Failed	https://keltinfreelow.atlassian.net/browse/S10-43
27	Verify the system behavior when sending an empty productsList array in the request body.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products	<pre>{ "productsList": [] }</pre> Click the "Send" button	- The response status code is 400 Bad Request - An error message in the response body indicating the product list cannot be empty	The response status code is 400 Bad Request, but the response body incorrectly states "No more than 30 items per set"	Failed	https://keltinfreelow.atlassian.net/browse/S10-44
28	Verify the user can successfully add products to a kit if the total number of items in the kit becomes 29.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products	The kit ID used to test 29 products is 7 <pre>{ "productsList": [{ "id": 1, "quantity": 1 }, { "id": 2, "quantity": 1 }, { "id": 3, "quantity": 1 }, { "id": 4, "quantity": 1 }, { "id": 5, "quantity": 1 }, { "id": 6, "quantity": 1 }, { "id": 7, "quantity": 1 }, { "id": 8, "quantity": 1 }, { "id": 9, "quantity": 1 }, { "id": 10, "quantity": 1 }, { "id": 11, "quantity": 1 }, { "id": 12, "quantity": 1 }, { "id": 13, "quantity": 1 }, { "id": 14, "quantity": 1 }, { "id": 15, "quantity": 1 }, { "id": 16, "quantity": 1 }, { "id": 17, "quantity": 1 }, { "id": 18, "quantity": 1 }, { "id": 19, "quantity": 1 }, { "id": 20, "quantity": 1 }, { "id": 21, "quantity": 1 }, { "id": 22, "quantity": 1 }, { "id": 23, "quantity": 1 }, { "id": 24, "quantity": 1 }, { "id": 25, "quantity": 1 }, { "id": 26, "quantity": 1 }, { "id": 27, "quantity": 1 }, { "id": 28, "quantity": 1 }, { "id": 29, "quantity": 1 }] }</pre> Click the "Send" button	- The response status code is 201 Created - The response body confirms 29 items were added to the kit.	The response status code is 200 OK. The JSON response body successfully displays the updated kit details, including the productsList array containing all 29 distinct product items.	Passed	
29	Verify the user can successfully add products to a kit if the total number of items in the kit becomes exactly 30.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products	The kit ID used to test 29 products is ID 7 <pre>{ "productsList": [{ "id": 1, "quantity": 1 }, { "id": 2, "quantity": 1 }, { "id": 3, "quantity": 1 }, { "id": 4, "quantity": 1 }, { "id": 5, "quantity": 1 }, { "id": 6, "quantity": 1 }, { "id": 7, "quantity": 1 }, { "id": 8, "quantity": 1 }, { "id": 9, "quantity": 1 }, { "id": 10, "quantity": 1 }, { "id": 11, "quantity": 1 }, { "id": 12, "quantity": 1 }, { "id": 13, "quantity": 1 }, { "id": 14, "quantity": 1 }, { "id": 15, "quantity": 1 }, { "id": 16, "quantity": 1 }, { "id": 17, "quantity": 1 }, { "id": 18, "quantity": 1 }, { "id": 19, "quantity": 1 }, { "id": 20, "quantity": 1 }, { "id": 21, "quantity": 1 }, { "id": 22, "quantity": 1 }, { "id": 23, "quantity": 1 }, { "id": 24, "quantity": 1 }, { "id": 25, "quantity": 1 }, { "id": 26, "quantity": 1 }, { "id": 27, "quantity": 1 }, { "id": 28, "quantity": 1 }, { "id": 29, "quantity": 1 }, { "id": 30, "quantity": 1 }] }</pre> Click the "Send" button	- The response status code is 201 Created - The response body confirms 30 items were added to the kit	The response status code is 200 OK. The JSON response body successfully displays the updated kit details, including the productsList array containing all 30 distinct product items.	Passed	
30	Verify the user cannot add products to a kit if the total number of items in the kit exceeds 30	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/:id/products	The kit ID used to test 29 products is ID 7 <pre>{ "productsList": [{ "id": 1, "quantity": 1 }, { "id": 2, "quantity": 1 }, { "id": 3, "quantity": 1 }, { "id": 4, "quantity": 1 }, { "id": 5, "quantity": 1 }, { "id": 6, "quantity": 1 }, { "id": 7, "quantity": 1 }, { "id": 8, "quantity": 1 }, { "id": 9, "quantity": 1 }, { "id": 10, "quantity": 1 }, { "id": 11, "quantity": 1 }, { "id": 12, "quantity": 1 }, { "id": 13, "quantity": 1 }, { "id": 14, "quantity": 1 }, { "id": 15, "quantity": 1 }, { "id": 16, "quantity": 1 }, { "id": 17, "quantity": 1 }, { "id": 18, "quantity": 1 }, { "id": 19, "quantity": 1 }, { "id": 20, "quantity": 1 }, { "id": 21, "quantity": 1 }, { "id": 22, "quantity": 1 }, { "id": 23, "quantity": 1 }, { "id": 24, "quantity": 1 }, { "id": 25, "quantity": 1 }, { "id": 26, "quantity": 1 }, { "id": 27, "quantity": 1 }, { "id": 28, "quantity": 1 }, { "id": 29, "quantity": 1 }, { "id": 30, "quantity": 1 }, { "id": 31, "quantity": 1 }] }</pre> Click the "Send" button	- The response status code is 400 Bad Request - An error message in the response body indicating the total number of items cannot exceed 30	The response status code is 400 Bad Request and the response body contains the message "No more than 30 items per set"	Passed	
31	Verify the user can add multiple distinct product objects inside the productsList array in a single request.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/6/products	<pre>{ "productsList": [{ "id": 1, "quantity": 1 }, { "id": 2, "quantity": 1 }] }</pre> Click the "Send" button	- The response status code is 200 OK - The response body confirms both distinct products were added successfully	the response status code is 200 OK. The JSON response body successfully displays the updated kit details for ID 6, including the productsList array.	Passed	
32	Verify the user can add the same product ID multiple times inside the same productsList array request.	1. Postman is open. 2. Set the request method to POST and enter the URL with a valid Kit ID: /api/v1/kits/5/products	<pre>{ "productsList": [{ "id": 1, "quantity": 1 }, { "id": 1, "quantity": 1 }] }</pre> Click the "Send" button	- The response status code is 200 OK - The response body confirms the product quantity was updated or added successfully	the response status code is 200 OK. The JSON response body successfully displays the updated kit details for ID 5, including the productsList array.	Passed	

[illegible]

Verify system behavior when deliveryTime value is 24	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>5.115</productsWeight> <deliveryTime>24</deliveryTime> </InputModel></pre> Click the "Send" button	- The response body returns an error message indicating 24 is an invalid hour value - Status: 400 Bad Request	- the response body returns an empty tag <response name="Fast Delivery"/> without any delivery status attributes - Status: 200 OK	Failed	https://keltinfreelow.atlassian.net/browse/S10-54
Verify that an error is returned if deliveryTime is a negative number.	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>5.115</productsWeight> <deliveryTime>-1</deliveryTime> </InputModel></pre> Click the "Send" button	- The response body returns an error message indicating -1 is a invalid hour value - Status: 400 Bad Request	- the response body returns an empty tag <response name="Fast Delivery"/> without any delivery status attributes - Status: 200 OK	Failed	https://keltinfreelow.atlassian.net/browse/S10-55
Verify that an error is returned if deliveryTime is empty.	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>5.115</productsWeight> <deliveryTime></deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code 400 Bad Request - The system blocks the deliveryTime parameter being empty	The response status code is 200 OK. The server accepts the empty tag and successfully calculates the delivery details, returning isItPossibleToDeliver="true" and the standard delivery time frame.	Failed	https://keltinfreelow.atlassian.net/browse/S10-58
Verify that if an error is returned if the deliveryTime is a invalid value type	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>5.115</productsWeight> <deliveryTime>nine</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code 400 Bad Request - The system blocks the deliveryTime parameter having a invalid value	The response status code is 200 OK. The server accepts the empty tag and successfully calculates the delivery details, returning isItPossibleToDeliver="true" and the standard delivery time frame.	Failed	https://keltinfreelow.atlassian.net/browse/S10-57
Verify that if an error is returned if the deliveryTime parameter is missing	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>5.115</productsWeight> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns a clear client error message indicating that deliveryTime is missing.	The system returns a 500 Internal Server Error. The response body displays a server crash message	Failed	https://keltinfreelow.atlassian.net/browse/S10-56
Verify that an error is returned if deliveryTime includes minutes.	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>5.115</productsWeight> <deliveryTime>7:30</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that minutes are not accepted.	The response status code is 200 OK. The server handles the invalid "7:30" input by ignoring the timestamp format, falling back to a default configuration, and returning a valid delivery calculation response.	Failed	https://keltinfreelow.atlassian.net/browse/S10-59
Verify that an error is returned if productsCount is a negative number.	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>-3</productsCount> <productsWeight>5.115</productsWeight> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that a negative number is not accepted.	- the response status code is 200 OK. The XML response body displays: - hostDeliveryCost: 6 - ClientDeliveryCost: 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-60
Verify that an error is returned if productsCount paramater is empty	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount></productsCount> <productsWeight>5.115</productsWeight> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that a empty value is not accepted.	- the response status code is 200 OK. The XML response body displays: - hostDeliveryCost: 3 - ClientDeliveryCost: 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-61
Verify that an error is returned if productsWeight is a negative number	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>-5.115</productsWeight> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that a negative number is not accepted.	- the response status code is 200 OK. The XML response body displays: - hostDeliveryCost: 3 - ClientDeliveryCost: 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-62
Verify that an error is returned if productsWeight parameter is empty	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight></productsWeight> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that a empty value is not accepted.	- the response status code is 200 OK. The XML response body displays: - hostDeliveryCost: 3 - ClientDeliveryCost: 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-63
Verify that an error is returned if productsWeight is a invalid value type	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <productsWeight>eight</productsWeight> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that an invalid value is not accepted	- the response status code is 200 OK. The XML response body displays: - hostDeliveryCost: 3 - ClientDeliveryCost: 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-64
Verify that if an error is returned if the productsWeight parameter is missing	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>2</productsCount> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that a parameter is missing	- the response status code is 500 internal server error - error message displays: cannot read properties of undefined reading 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-65
Verify that if an error is returned if productsCount is a invalid value type	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsCount>two</productsCount> <productsWeight>5.115</productsWeight> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that a invalid value is not accepted	- the response status code is 200 OK. The XML response body displays: - hostDeliveryCost: 3 - ClientDeliveryCost: 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-66
Verify that if an error is returned if productsCount parameter is missing	1. Postman is open. 2. Set the request method to POST and enter the URL + Endpoint: /fast-delivery/v3.1.1/calculate-delivery.xml 3. Change the request Body type to 'raw' and select 'XML' from the format dropdown	<pre><InputModel> <productsWeight>5.115</productsWeight> <deliveryTime>20</deliveryTime> </InputModel></pre> Click the "Send" button	- The response status code is 400 Bad Request. - The response body returns an error message indicating that a parameter is missing	- the response status code is 500 internal server error - error message displays: cannot read properties of undefined reading 0	Failed	https://keltinfreelow.atlassian.net/browse/S10-67

[illegible]